Correções do backend (Parte 1 (manutenção)) — guia para leigo

PS Adapti 2026.2 · 4ª etapa · material de estudo do Thilon

Este PDF explica **tudo o que mudou no Laravel**, em linguagem de quem está aprendendo. Em cada correção você vai ver:

1. o que é aquele arquivo (para que ele serve);
2. o que estava quebrado, em português claro;
3. **o caminho do arquivo e as linhas exatas** no código de hoje;
4. o que a linha faz, palavra por palavra quando necessário.

Números de linha batem com o código **atual** do projeto. Se você editar o arquivo depois, a linha pode "descer" uns números — o trecho continua o mesmo.

O que este documento NÃO cobre: telas do Next.js, Tarefas, trava de 2 projetos ativos. Só a API de Projetos e Membros.

Pasta base: `ps-desafio-2026-2/backend/`

---

0. Mini glossário (leia isto primeiro)

O Laravel é o "cérebro" do servidor. O Next.js (frontend) manda um pedido HTTP; o Laravel responde JSON.

Palavra	O que significa, sem firula
API	Porta de entrada do backend. Alguém manda um pedido (`POST /projetos`) e recebe um JSON de volta.
Rota	O endereço da API. Ex.: `POST /membros` = "criar membro". Fica em `routes/api.php`.
Controller	Arquivo que **executa** a ação: buscar, criar, atualizar, apagar. Ele não deveria validar regra de negócio pesada.
Form Request	"Porteiro". Antes do controller gravar, este arquivo pergunta: os dados são válidos? Se não, a API responde **422** e o banco nem é tocado.
Model	Representa uma tabela. `Membro` = tabela `membros`. É ele quem fala com o banco.
Migration	Receita para **criar/alterar tabelas**. O banco só fica do jeito certo depois de rodar `php artisan migrate`.
Factory / Seeder	Geram dados falsos para testar (10 projetos, 40 membros). Não são a API em si.
`$fillable`	Lista de colunas que o Eloquent **autoriza** a gravar de uma vez (`create($dados)`). Campo fora dessa lista é **ignorado**.
`$hidden`	Campo que existe no banco mas **não aparece** no JSON (ex.: senha).
Cast `hashed`	Na hora de salvar senha, o Laravel transforma em hash (embaralha). Ninguém lê a senha original depois.

Mass assignment	Gravar vários campos de uma vez com `create()` / `update()`. Só entra o que está em `$fillable`.
HTTP 403	"Proibido". Aqui acontecia porque `authorize()` retornava `false`.
HTTP 422	"Entendi o pedido, mas os dados estão errados." É o status certo para validação (data invertida, e-mail duplicado).
HTTP 201	Criado com sucesso.
`unique`	Não pode repetir. Dois membros com o mesmo e-mail = erro.
`nullable`	A coluna pode ficar vazia (`NULL`). Se não for nullable, o banco **obriga** um valor e explode SQL.
`sometimes`	Na atualização: "se o campo vier no pedido, valide; se não vier, ignore". Serve para PATCH parcial.

Caminho de um pedido (guarde isso)

```
Frontend (ou Postman)
    → rota em routes/api.php
        → Form Request (authorize + rules)     ← a maior parte das correções está aqui
            → Controller (grava / lê)
                → Model ($fillable, casts)
                    → Banco (migration define as colunas)
```

Se o porteiro (`authorize`) diz não → **403**, controller nem roda.
Se as regras (`rules`) falham → **422**, banco não é escrito.
Se o model ignora um campo (`fillable`) → o banco grava **sem aquele campo**, mesmo o porteiro tendo aceitado.

---

1. Mapa rápido: o que estava errado

Bug na vida real	Arquivo	Linhas de hoje	O que a API fazia
Projeto com fim antes do início	`StoreProjetoRequest.php`	**21**	Aceitava e gravava
Editar projeto sempre proibido	`UpdateProjetoRequest.php`	**11–13** (antes `return false`)	Sempre **403**
Editar só a data fim furava a trava	`UpdateProjetoRequest.php`	**27–51** (`withValidator`)	Podia gravar período invertido
Criar/editar membro sempre proibido	`StoreMembroRequest.php` **10–13**; `UpdateMembroRequest.php` **11–14**	Sempre **403**	
Senha validada e jogada fora	`Membro.php`	**16** (`'senha'` no fillable)	INSERT sem senha / SQL error

Senha em texto puro	`Membro.php`	**26–32**	Risco de gravar senha legível
E-mail repetido / inválido	Store **19**; Update **20–27**; migration **17**	Dois membros iguais	
Cor/aniversário "opcionais" no PHP e obrigatórios no banco	migration **19–20**	Erro SQL feio, não 422	
Seed de projeto com data bagunçada	`ProjetoFactory.php`	**19–23** (`clone`)	Dados de teste mentiam
Rotas HTML de projeto sem tela	`api.php`	**24**	`create`/`edit` inúteis

---

2. Projetos — criar (`StoreProjetoRequest`)

Arquivo: `backend/app/Http/Requests/StoreProjetoRequest.php`
Para que serve: porteiro do `POST /projetos`. O controller `ProjetoController::store` (arquivo `ProjetoController.php`, **linhas 31–34**) só grava *depois* deste porteiro.

2.1 `authorize()` — linhas 9–12

```php
public function authorize(): bool
{
    return true;    // linha 11
}
```

`true` = "pode tentar cadastrar". Já era `true` neste arquivo; **não era o bug**. O bug de 403 estava no *update* (próxima seção).

2.2 `rules()` — linhas 14–23 (aqui estava o bug do edital)

Antes (ideia):

```php
'data_inicio' => 'required|date',
'data_fim'    => 'required|date',
```

Isso só pergunta: "é uma data?". `2026-12-01` início e `2026-01-01` fim passavam.

Agora, linha 21 — a trava:

```php
'data_fim' => ['required', 'date', 'after_or_equal:data_inicio'],
```

Tradução das três palavras:

- `required` = obrigatório;
- `date` = tem que parecer data;
- **`after_or_equal:data_inicio`** = tem que ser **igual ou depois** do campo `data_inicio`.

Também nas linhas **17–19**: `string` e `max:255` (nome/cliente não podem ser um livro).

2.3 Nomes bonitos na mensagem — `attributes()`, linhas 25–34

O Laravel, por padrão, diria "The data fim field...". `attributes()` troca o nome técnico pelo nome em português (`data de término`).

2.4 Mensagem clara — `messages()`, linhas 36–41, **linha 39**

```
'data_fim.after_or_equal' => 'A data de término deve ser igual ou posterior à data de início.',
```

A chave `data_fim.after_or_equal` significa: "quando a regra `after_or_equal` do campo `data_fim` falhar, use este texto". A API devolve **422** com esse texto em `errors.data_fim`.

---

3. Projetos — atualizar (`UpdateProjetoRequest`)

Arquivo: `backend/app/Http/Requests/UpdateProjetoRequest.php` (73 linhas hoje)
Usado por: `ProjetoController.php` método `update`, **linhas 49–53**.

Este arquivo estava **quase vazio e trancado**. Era o segundo bug grave.

3.1 Imports novos — linhas 5–6

```
use App\Models\Projeto;
use Illuminate\Contracts\Validation\Validator;
```

Precisamos do **model** `Projeto` para buscar no banco a data antiga (linha 34). Precisamos de `Validator` para o `withValidator` (linha 27).

3.2 `authorize()` — linhas 11–14 ★ bug 403

Antes: `return false;`
Agora, linha 13: `return true;`

Analogia: o porteiro do *update* estava com a porta trancada para todo mundo. O Laravel nem lia as regras: respondia **403 Forbidden**. Por isso "editar projeto" no Postman/frontend morria sem mensagem de validação.

3.3 `rules()` — linhas 16–25

Antes: `return [];` (zero regras).
Agora: as mesmas ideias do store, com `sometimes` na frente.

`sometimes` (linhas 19–23) = "se o campo **vier** no JSON, aí sim é obrigatório e tem que ser válido. Se não veio, não reclama." Isso permite mandar só `nome` sem reenviar as datas.

Nas linhas **22–23** as datas **não** usam `after_or_equal` aqui. Motivo: no PATCH você pode mandar **só** `data_fim`. A regra `after_or_equal:data_inicio` olharia um `data_inicio` **ausente no payload**, não o valor do banco. A comparação completa está no bloco seguinte.

3.4 `withValidator()` — linhas 27–51 ★ o detalhe extra

Leia em voz alta:

Linha	O que acontece
27	"Depois das regras simples, rode um extra."
29–32	Se já tem erro (ex.: data malformada), **não** compara. Evita mensagem duplicada.
34	Busca o projeto pelo id da URL. O nome da rota é `projeto` (vem do `apiResource`).
36–39	`$inicio` = o que veio no pedido; se não veio, usa a data **já salva**.
40–43	Idem para `$fim`.
45–50	Se as duas existem e fim < início, adiciona erro no campo `data_fim` (422).

`$this->route('projeto')` não é magia: é o `{projeto}` da URL `/projetos/abc-uuid`.

3.5 `attributes()` — linhas 54–63

Igual ao store: nomes em português.

3.6 `messages()` — linhas 65–71

Linhas **68–69**: se alguém mandar `data_inicio=banana`, a API diz "Informe uma data de início válida." em vez de inglês genérico.

---

4. O controller de Projeto (quase não mudou)

Arquivo: `backend/app/Http/Controllers/ProjetoController.php`

Não precisamos reescrever a lógica de gravar. Ele já fazia o certo:

- **store, linhas 31–34:** `Projeto::create($request->validated())` — só entra o que o porteiro aprovou;
- **update, linhas 49–53:** `findOrFail` + `update($request->validated())`.

O bug **não** estava no controller. Estava no porteiro (`UpdateProjetoRequest` 403 + regras vazias) e na falta de `after_or_equal` no store.

---

5. Membros — criar (`StoreMembroRequest`)

Arquivo: `backend/app/Http/Requests/StoreMembroRequest.php`
Usado por: `MembroController.php` método `store`, **linhas 34–44**.

5.1 `authorize()` — linhas 10–13 ★ 403

Antes: `return false;` → criar membro era impossível.
Agora, linha 12: `return true;`

5.2 E-mail — linha 19 ★ duplicata e lixo

```
'email' => ['required', 'string', 'email', 'max:255', 'unique:membros,email'],
```

Pedacinho	Significado
`required`	tem que vir
`string`	texto
`email`	tem que parecer e-mail (`a@b.com`). Antes **não tinha** isso
`max:255`	tamanho da coluna
`unique:membros,email`	procura na tabela `membros`, coluna `email`. Se já existe, 422

5.3 Senha — linhas 20–29

A Password Rule (linhas **24–28**) já existia no template: mínimo 8, letra, maiúscula+minúscula, número, símbolo. Exemplo que passa: `Password1!`.

O problema da senha **não** era esta regra. Era o model não gravar o campo (seção 7).

5.4 Cor e aniversário — linhas 30–31

`nullable` = pode omitir. Isso **já batia** com o formulário do frontend. Quem não batia era a **tabela** (seção 8).

5.5 `attributes()` — linhas 36–46

Troca `email` por "e-mail" nas mensagens.

5.6 `messages()` — linhas 48–54, **linhas 51–52**

- unique → "Já existe um membro cadastrado com este e-mail."
- email inválido → "Informe um e-mail válido."

---

6. Membros — atualizar (`UpdateMembroRequest`)

Arquivo: `backend/app/Http/Requests/UpdateMembroRequest.php`

6.1 Import extra — linha 6

```php
use Illuminate\Validation\Rule;
```

Serve para montar unique **ignorando o próprio membro** (senão, ao salvar sem mudar o e-mail, o Laravel diria "e-mail já existe" — o dele mesmo).

6.2 `authorize()` — linhas 11–14 ★ 403

Antes: `false`. **Agora linha 13:** `true`.

6.3 Unique com ignore — linhas 20–27, **linha 26**

```php
Rule::unique('membros', 'email')->ignore($this->route('membro')),
```

`$this->route('membro')` = o UUID na URL `/membros/aquele-id`. "Este id pode continuar com o e-mail dele; qualquer **outro** registro não."

O nome `membro` vem do `apiResource('/membros')` (Laravel usa o nome do recurso no singular).

6.4 Resto das rules — linhas 19, 28–41

`sometimes` = PATCH parcial. Senha só é revalidada se vier no pedido (linhas 28–38). Cor/aniversário podem ser nulos (linhas 39–40).

6.5 Mensagens — linhas 57–63

Iguais às do store (e-mail único / e-mail inválido).

---

7. Model `Membro` — o coração da inconsistência

Arquivo: `backend/app/Models/Membro.php`

Aqui o edital falava em "inconsistências no banco". Não era o MySQL "bugado". Era o PHP **não combinando** com a tabela.

7.1 `$fillable` — linhas 13–20, **linha 16 ★ senha sumia**

Antes a lista era: `nome`, `email`, `cor_favorita`, `data_aniversario`, `image`. **Sem** `senha`.

O controller (linha 43 do `MembroController`) faz:

```
$membro = $this->membro->create($validatedData);
```

O porteiro *tinha* validado `senha`. O `create()` **descarta** o que não está em fillable. A coluna `senha` no banco é NOT NULL → ou o INSERT explodia, ou nascia um registro sem senha útil.

Agora a linha 16 é `'senha'`. A senha entra no INSERT.

Analogia: o porteiro colocou a senha na bandeja; o garçom (`fillable`) não tinha "senha" no cardápio e jogava a bandeja fora.

7.2 `$hidden` — linhas 22–24 (não mudou a ideia)

```
protected $hidden = [
    'senha',  // linha 23
];
```

A senha **grava**, mas o JSON de resposta **não mostra**. Você testa isso no Postman: 201 e o corpo vem sem `senha`.

7.3 `casts()` — linhas 26–32 ★ hash

Antes: não existia este método.

```
'senha' => 'hashed',            // linha 29
'data_aniversario' => 'date',   // linha 30
```

`hashed` = na hora do `set`, o Laravel aplica `Hash::make`. Ninguém precisa hashear no controller.

`date` = trata aniversário como data (Carbon), não string solta.

Por que a factory não pode mandar bcrypt pronto: se a factory já hashear e o model hashear de novo, vira hash-de-hash. Por isso a factory manda texto (`Password1!`). Ver seção 9.

---

8. Migration da tabela `membros`

Arquivo: `backend/database/migrations/2026_10_06_234716_create_membros_table.php`
Método `up()` : linhas **12–24** (é o "criar tabela").

Migration = receita do banco. Validar no PHP **não altera** coluna sozinha. Os dois têm que combinar.

8.1 E-mail unique — linha 17

```
$table->string('email')->unique();
```

Antes: `$table->string('email');` (podia repetir no banco mesmo se o PHP tentasse impedir). Índice unique é a **última trava**: mesmo um bug no PHP não deixa dois iguais.

8.2 Cor e aniversário nullable — linhas 19–20

```
$table->string('cor_favorita')->nullable();
$table->date('data_aniversario')->nullable();
```

Antes: sem `nullable()` = NOT NULL. Request dizia "pode omitir" → MySQL respondia erro SQL (`SQLSTATE 23000`), não 422 bonito.

Decisão: banco acompanha o formulário (campos opcionais). Nome, e-mail e senha continuam obrigatórios.

8.3 Por que não tem um segundo arquivo ALTER

`->change()` no Laravel 11 deste template pede pacote extra (`doctrine/dbal`). Por isso a correção foi **na migration de create**.

Se o seu PC **já** rodou a migration antiga, o banco local ainda está velho. Aí, **só em local**:

```
cd backend
php artisan migrate:fresh --seed
```

Isso **apaga** as tabelas e recria. Não use em produção.

---

9. Factories (dados de teste)

Factories não são a API. Elas alimentam `php artisan db:seed`. Mesmo assim, seed podre atrapalha demo.

9.1 `ProjetoFactory.php` — linhas 19–23 ★ clone

Arquivo: `backend/database/factories/ProjetoFactory.php`

Antes (ideia): `$dataInicio->modify('+1 day')` *dentro* do `dateTimeBetween`. Em PHP, `DateTime::modify` **muda o próprio objeto**. Depois, `data_inicio` já era o dia seguinte.

Agora, linha 21:

```
(clone $dataInicio)->modify('+1 day'),
```

`clone` = cópia. A original (linha 19) continua o início verdadeiro. Linhas **29–30** gravam `Y-m-d`.

9.2 `MembroFactory.php` — linha 22

Arquivo: `backend/database/factories/MembroFactory.php`

```
'senha' => 'Password1!',
```

Texto puro de propósito (cast `hashed` no model). `Password1!` passa na Password Rule se alguém usar a factory em teste de validação. Linha **21** já tinha `unique()` no e-mail do Faker.

---

10. Rotas — `routes/api.php`

Arquivo: `backend/routes/api.php`

Linha 23 (membros) — já era `apiResource` (ajustamos só a barra `/membros`).
Linha 24 (projetos) ★

Antes: `Route::resource('projetos', ...)`
Agora:

```
Route::apiResource('/projetos', \App\Http\Controllers\ProjetoController::class);
```

`resource` completo registra também:

- `GET /projetos/create`
- `GET /projetos/{id}/edit`

Isso é para página HTML (Blade). Este projeto é **API JSON**. Essas duas rotas extra não têm tela. `apiResource` deixa só: index, store, show, update, destroy — o que o Next usa.

Observação (não mudamos de propósito): essas rotas **não** estão dentro do `auth:sanctum` (linhas 9–17). O template original deixou Projetos/Membros públicos. Não misturamos isso nesta etapa.

Prefixo da API neste projeto é **vazio** (`bootstrap/app.php`, `apiPrefix: ''`). Então a URL real é `http://127.0.0.1:8000/projetos`, **sem** `/api`.

---

11. `MembroController` — mudança mínima

Arquivo: `backend/app/Http/Controllers/MembroController.php`

A lógica de foto (linhas **38–41** no store, **67–75** no update) **não mudou**.

Única limpeza: era um `use Throwable;` no topo que **não era usado**. Removido. Hoje o arquivo termina os `use` na **linha 10**.

Não hasheamos senha aqui. O model (linha 29 de `Membro.php`) faz isso quando o `create` da **linha 43** e o `update` da **linha 77** rodam.

---

12. Lista fechada de arquivos (para abrir no Cursor)

Caminhos a partir de `ps-desafio-2026-2/`:

1. `backend/app/Http/Requests/StoreProjetoRequest.php` — linhas **17–21**, **25–41**
2. `backend/app/Http/Requests/UpdateProjetoRequest.php` — **5–6**, **13**, **16–25**, **27–51**, **54–71**
3. `backend/app/Http/Requests/StoreMembroRequest.php` — **12**, **18–32**, **36–54**
4. `backend/app/Http/Requests/UpdateMembroRequest.php` — **6**, **13**, **19–41**, **45–63**
5. `backend/app/Models/Membro.php` — **16**, **22–24**, **26–32**
6. `backend/app/Models/Projeto.php` — **21–27** (casts)
7. `backend/database/migrations/2026_10_06_234716_create_membros_table.php` — **17**, **19**, **20**
8. `backend/database/factories/MembroFactory.php` — **22**
9. `backend/database/factories/ProjetoFactory.php` — **19–23**
10. `backend/routes/api.php` — **23–24**
11. `backend/app/Http/Controllers/MembroController.php` — só sumiu o `use Throwable`

No Cursor: Ctrl+P, cola o caminho, depois Ctrl+G e digita o número da linha.

---

13. Model Projeto — detalhe extra

Arquivo: `backend/app/Models/Projeto.php`, **linhas 21–27**

```php
protected function casts(): array
{
    return [
        'data_inicio' => 'date',
        'data_fim' => 'date',
    ];
}
```

Não era o bug do edital. Faz o Eloquent tratar as colunas como data. O `withValidator` usa isso na **linha 38** de `UpdateProjetoRequest` (`->format('Y-m-d')`).

---

14. Como testar (Postman / Insomnia)

1. Pasta `backend`: `php artisan migrate:fresh --seed` (local!) e `php artisan serve`.
2. Base: `http://127.0.0.1:8000`.

O que você faz	O que deve acontecer	Prova que linha
`POST /projetos` com fim antes do início	**422**, texto da data de término	Store, linha **21** e **39**
`POST /projetos` com datas ok	**201**	Store + `ProjetoController` 31–34
`PUT /projetos/{id}` válido	**200** (antes 403)	Update, linha **13**

`PATCH /projetos/{id}` só com `data_fim` anterior ao início já salvo	**422**	Update, linhas **27–51**
`POST /membros` válido (`Password1!`)	**201**, JSON **sem** `senha`	Store linha **12**; Model **16**, **23**, **29**
`POST /membros` e-mail repetido	**422** unique	Store linha **19**; migration **17**
`POST /membros` sem cor/aniversário	**201**	migration **19–20**
`PUT /membros/{id}` válido	**200** (antes 403)	UpdateMembro linha **13**

---

15. Frases prontas para a apresentação

- "Form Request é o porteiro. Controller não valida data; `StoreProjetoRequest` linha 21 tem `after_or_equal`."
- "Update de projeto dava 403 porque `authorize` era false na linha 13 de `UpdateProjetoRequest`. Não era peretapa de usuário: era o arquivo de request trancado."
- "PATCH só com data fim usa `withValidator` linhas 27–51, porque `after_or_equal` não enxerga o banco."
- "Senha: validada no request, descartada no model. Mass assignment. Fillable linha 16 de `Membro.php`."
- "Hash: cast `hashed` linha 29, igual o User do template. `$hidden` linha 23 esconde no JSON."
- "Unique em dois andares: PHP linha 19 do Store e índice na migration linha 17. Se um falhar, o outro segura."
- "Cor e aniversário: PHP dizia nullable, tabela dizia NOT NULL. Alinhei a migration linhas 19–20."

---

16. O que propositalmente ficou para depois

- Frontend (tabela no celular, hidratação, toasts).
- Tarefas, pivot membro–projeto, trava de 2 projetos ativos (Parte 2 (tarefas)).
- Sanctum nessas rotas (o template deixou público).
- README / prints da entrega (etapa 3).

A Parte 2 (tarefas) vai **depender** desta base: se create de membro ainda fosse 403, tarefa atribuída a membro nem nascia.

---

17. Recado final

Se perguntarem "por que não validar no controller?": **controller orquestra, Form Request valida entrada, Model protege persistência** (`fillable` , casts, unique no banco). O código base misturava isso ou deixava pela metade. Cada correção neste PDF é um desses três papéis sendo colocado no lugar certo, com arquivo e linha para você apontar na tela.